# Code Clarification for Sahith's Tabak-Turner Implementation

James Evans

08/08/2025

```python
def estimate_forward_KL(X, flow, log_rho_exact):
    # compute log-density under the exact model
    log_ex = log_rho_exact(X)

    # compute log-density under the flow estimate
    log_est = flow.log_prob(X)

    # Monte Carlo KL
    kl_mc = np.mean(log_ex - log_est)
    return kl_mc
```

## Forward KL Divergence Estimation

We consider the forward Kullback–Leibler divergence from the true distribution $\rho$ to the model $\hat{\rho}$:

$$\mathrm{KL}(\rho \,\|\, \hat{\rho}) \;=\; \int_{\mathbb{R}^n} \rho(x)\, \log \frac{\rho(x)}{\hat{\rho}(x)}\, dx.$$

Expanding the logarithm, we can rewrite this as:

$$\mathrm{KL}(\rho \,\|\, \hat{\rho}) = \int_{\mathbb{R}^n} \rho(x) \big[ \log \rho(x) - \log \hat{\rho}(x) \big]\, dx.$$

### Monte Carlo Approximation

If $X_1, \ldots, X_m \overset{\text{i.i.d.}}{\sim} \rho$, then by the law of large numbers we have:

$$\frac{1}{m} \sum_{i=1}^{m} \big[ \log \rho(X_i) - \log \hat{\rho}(X_i) \big] \;\xrightarrow[m\to\infty]{}\; \mathrm{KL}(\rho \,\|\, \hat{\rho}).$$

Therefore, a Monte Carlo estimator of the forward KL divergence is given by:

$$\widehat{\mathrm{KL}}(\rho \,\|\, \hat{\rho}) \;=\; \frac{1}{m} \sum_{i=1}^{m} \big[ \log \rho(X_i) - \log \hat{\rho}(X_i) \big].$$

### Interpretation

- The term $\log \rho(X_i)$ is the exact log-density of the true distribution evaluated at the sample $X_i$.
- The term $\log \hat{\rho}(X_i)$ is the log-density predicted by the model at the same sample.
- The difference $\log \rho(X_i) - \log \hat{\rho}(X_i)$ measures, for that sample, how much more likely it is under the true distribution than under the model.

- Averaging over all samples approximates the expectation $\mathbb{E}_\rho[\,\cdot\,]$ and yields the KL estimate.

```python
def _volume_n_ball(self, n):
    """Calculates the volume of an n-dimensional unit ball."""
    return np.pi**(n/2) / gamma(n/2 + 1)

def _calculate_alpha(self, x0, n, m):
    """
    Calculates the length-scale 'alpha' for a map centered at x0,
    based on equation (13) from the paper. The goal is to have the
    map's influence cover roughly n_p points.

    Args:
        x0 (np.ndarray): The center of the elementary map.
        n (int): The dimensionality of the data space.
        m (int): The total number of data points.

    Returns:
        float: The calculated alpha.
    """
    omega_n = self._volume_n_ball(n)
    # The formula in the paper is derived for a target normal distribution.
    # It adapts the map's radius to be larger in low-density regions.
    term = (omega_n**-1 * self.n_p / m)**(1/n)
    alpha = (2 * np.pi)**0.5 * term * np.exp(np.linalg.norm(x0)**2 / (2*n))
    return alpha
```

# Role of $\rho$ and $\mu$ in the Algorithm

In the training phase, every point provided to `fit(..)` is assumed to be sampled from the *source density* $\rho$:

$$x_i \sim \rho.$$

The algorithm learns a transformation that maps $\rho$ to the *target density* $\mu$.

## Pipeline Overview

**Input:** Start with $x_i \sim \rho$.

**Preconditioning:** We transform each $x_i$ to

$$z_i = \frac{x_i - \bar{x}}{\text{std}},$$

where $\bar{x}$ is the empirical mean and std is a global scaling factor chosen so that the cloud is roughly isotropic. These $z_i$ are still samples from a distribution linearly related to $\rho$.

**Forward Maps $\rho \to \mu$:** The algorithm builds a composition of small, localized *radial maps* that progressively transform $z_i$ to resemble the target Gaussian $\mu = \mathcal{N}(0, I_n)$.

**Final Map:** The learned transformation $T$ satisfies

$$y_i = T(x_i) \quad \text{with} \quad y_i \sim \mu,$$

and, by the change-of-variables formula,

$$\hat{\rho}(x) = \mu\big(T(x)\big)\,\big|\det \nabla T(x)\big|. \quad \text{Jacobian}$$

## Mathematical Role of `volume_n_ball` and `calculate_alpha`

The algorithm builds the transport map as a composition of *elementary radial maps* centered at points $x_0$. The *length scale* $\alpha$ for a given map determines its **region of influence**. We choose $\alpha$ so that the ball $B(x_0, \alpha)$ contains, on average, $n_p$ samples from the *source* distribution $\rho$ (in preconditioned coordinates).

**`volume_n_ball`** This helper function returns the volume of the $n$-dimensional unit ball:

$$\omega_n = \frac{\pi^{n/2}}{\Gamma\left(\frac{n}{2}+1\right)}.$$

For any radius $r > 0$, the $n$-dimensional volume is:

$$\mathrm{Vol}\big(B(0,r)\big) = \omega_n r^n.$$

**Small-ball mass requirement**

Let $\phi_n(z)$ be the $n$-dimensional standard Gaussian density:

$$\phi_n(z) = (2\pi)^{-n/2} \exp\left(-\frac{\|z\|^2}{2}\right).$$

In preconditioned space, the *target* $\mu$ is $\mathcal{N}(0, I_n)$, so the algorithm estimates $\alpha$ assuming the point cloud is drawn from this $\mu$.

The condition "$B(x_0, \alpha)$ contains $n_p$ points on average" is:

$$m \int_{B(x_0,\alpha)} \phi_n(z)\, dz = n_p,$$

where $m$ is the total number of points.

**Local density approximation**

If $\alpha$ is small compared to the scale over which $\phi_n$ changes, we approximate:

$$\phi_n(z) \approx \phi_n(x_0) \quad \text{for } z \in B(x_0, \alpha).$$

Thus:

$$\int_{B(x_0,\alpha)} \phi_n(z)\, dz \;\approx\; \phi_n(x_0) \cdot \mathrm{Vol}\big(B(x_0,\alpha)\big) \;=\; \phi_n(x_0) \cdot \omega_n \alpha^n.$$

**Substitution into the mass equation**

The mass equation becomes:

$$m \cdot \phi_n(x_0) \cdot \omega_n \alpha^n = n_p.$$

Using the Gaussian density at $x_0$:

$$\phi_n(x_0) = (2\pi)^{-n/2} \exp\left(-\frac{\|x_0\|^2}{2}\right),$$

we have:

$$m \cdot (2\pi)^{-n/2} \exp\left(-\frac{\|x_0\|^2}{2}\right) \cdot \omega_n \alpha^n = n_p.$$

**Solve for $\alpha$**

Rearranging:

$$\alpha^n = \frac{n_p}{m\,\omega_n} \cdot (2\pi)^{n/2} \cdot \exp\left(\frac{\|x_0\|^2}{2}\right).$$

Taking the $n$-th root:

$$\alpha = \left(\frac{n_p}{m\,\omega_n}\right)^{1/n} \cdot (2\pi)^{1/2} \cdot \exp\left(\frac{\|x_0\|^2}{2n}\right).$$

## Why the Small-Ball Mass Equation Represents an Expectation

We have $m$ sample points $z^{(1)}, z^{(2)}, \ldots, z^{(m)}$ drawn i.i.d. from the *target* density $\phi_n(z)$. Define the indicator random variable:

$$X_i = \begin{cases} 1 & \text{if } z^{(i)} \in B(x_0, \alpha), \\ 0 & \text{otherwise.} \end{cases}$$

By the definition of expectation for an indicator variable:

$$\mathbb{E}[X_i] = \mathbb{P}\Big(z^{(i)} \in B(x_0, \alpha)\Big) = \int_{B(x_0,\alpha)} \phi_n(z)\,dz.$$ Probability mass

Now let $N_{\text{in}}$ be the total number of points in the ball:

$$N_{\text{in}} = \sum_{i=1}^{m} X_i.$$

By linearity of expectation:

$$\mathbb{E}[N_{\text{in}}] = \sum_{i=1}^{m} \mathbb{E}[X_i] = m \int_{B(x_0,\alpha)} \phi_n(z)\,dz.$$

Thus, the quantity

$$m \int_{B(x_0,\alpha)} \phi_n(z)\,dz$$

is exactly the *expected number of sample points* inside the ball $B(x_0, \alpha)$.

```python
def _radial_f(self, r, alpha):
    """
    The radial localization function f(r), based on equation (24).
    Handles the r=0 case to avoid division by zero.

    Args:
        r (np.ndarray): Array of radial distances ||x - x0||.
        alpha (float): The length-scale of the map.

    Returns:
        np.ndarray: The value of the localization function.
    """
    # Handle r=0 separately to avoid division by zero.
    # The limit of erf(x)/x as x->0 is 2/sqrt(pi).
    f_vals = np.zeros_like(r)
    nonzero_r = r != 0
    zero_r = ~nonzero_r

    r_scaled = r[nonzero_r] / alpha
    f_vals[nonzero_r] = erf(r_scaled) / r[nonzero_r]
    f_vals[zero_r] = 2.0 / (alpha * np.sqrt(np.pi))
    return f_vals
```

# Mathematical Role of `_radial_f`

Let $x_0 \in \mathbb{R}^n$ be the center of an elementary radial map, and let

$$r = \|x - x_0\|$$

denote the Euclidean distance from $x$ to $x_0$. Suppose the potential $F$ is purely radial, i.e. $F(x) = \Phi(r)$ for some scalar function $\Phi$. Then by the chain rule,

$$\nabla F(x) = \frac{d\Phi}{dr} \cdot \frac{x - x_0}{r}.$$

The scalar function

$$f(r) := \frac{d\Phi}{dr}$$

is called the *radial localization function*. In the algorithm, $f(r)$ is defined (based on equation (24) in the paper) as

$$f(r;\alpha) = \begin{cases} \dfrac{\operatorname{erf}\left(\frac{r}{\alpha}\right)}{r}, & r > 0, \\ \dfrac{2}{\alpha\sqrt{\pi}}, & r = 0, \end{cases}$$

where $\alpha > 0$ is the length scale of the map.

**Special case $r = 0$.** Directly evaluating $\operatorname{erf}(r/\alpha)/r$ at $r = 0$ causes a division-by-zero. Instead, we take the limit:

$$\lim_{r\to 0} \frac{\operatorname{erf}(r/\alpha)}{r}.$$

Using the Taylor expansion $\operatorname{erf}(t) \approx \frac{2}{\sqrt{\pi}}\, t$ for small $t$, we find:

$$\lim_{r\to 0} \frac{\operatorname{erf}(r/\alpha)}{r} = \lim_{r\to 0} \frac{\frac{2}{\sqrt{\pi}}(r/\alpha)}{r} = \frac{2}{\alpha\sqrt{\pi}},$$

which matches the $r = 0$ branch above.

### Interpretation

- For $r > 0$, $f(r;\alpha)$ decays roughly like $1/r$ for large $r$, localizing the influence of the map.
- For $r = 0$, the finite value $\frac{2}{\alpha\sqrt{\pi}}$ ensures smooth behavior at the center.
- The gradient of the potential is then given by $\nabla F(x) = f(r;\alpha)\,(x - x_0)/r$.

# Differentiating the Radially Symmetric Potential

We are given the function:

$$F_\ell(r) = r \cdot \operatorname{erf}\left(\frac{r}{\alpha}\right) + \frac{\alpha}{\sqrt{\pi}} e^{-(r/\alpha)^2}$$

We aim to compute the derivative with respect to $r$, that is,

$$\frac{dF_\ell}{dr}$$

### Differentiate the first term

$$\frac{d}{dr}\left(r \cdot \operatorname{erf}\left(\frac{r}{\alpha}\right)\right)$$

Using the product rule:

$$= \operatorname{erf}\left(\frac{r}{\alpha}\right) + r \cdot \frac{d}{dr}\left[\operatorname{erf}\left(\frac{r}{\alpha}\right)\right]$$

Recall the derivative of the error function:

$$\frac{d}{dx}\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}}e^{-x^2} \quad \Rightarrow \quad \frac{d}{dr}\operatorname{erf}\left(\frac{r}{\alpha}\right) = \frac{2}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2}$$

So the full derivative of the first term is:

$$\operatorname{erf}\left(\frac{r}{\alpha}\right) + \frac{2r}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2}$$

### Differentiate the second term

$$\frac{d}{dr}\left(\frac{\alpha}{\sqrt{\pi}}e^{-(r/\alpha)^2}\right) = \frac{\alpha}{\sqrt{\pi}} \cdot \left(-\frac{2r}{\alpha^2}\right)e^{-(r/\alpha)^2} = -\frac{2r}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2}$$

### Combine the results

The exponential terms from both derivatives cancel:

$$\frac{2r}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2} - \frac{2r}{\alpha\sqrt{\pi}}e^{-(r/\alpha)^2} = 0$$

So we are left with:

$$\frac{dF_\ell}{dr} = \operatorname{erf}\left(\frac{r}{\alpha}\right)$$

## Radial Function Definition

We consider scalar potentials $F(x)$ of the form:

$$F(x) = \Phi(\|x - \bar{x}\|),$$

where:

- $\bar{x} \in \mathbb{R}^d$ is a fixed center point (e.g., sampled from the source distribution $\rho$),
- $r = \|x - \bar{x}\|$ is the radial distance from $x$ to $\bar{x}$,
- $\Phi : \mathbb{R}_{\geq 0} \to \mathbb{R}$ is a smooth convex scalar function,
- $F(x)$ is a radially symmetric and convex function.

### Gradient of the Potential

To compute $\nabla F(x)$, we apply the chain rule:

$$\begin{aligned}\nabla F(x) &= \frac{d}{dr}\Phi(r) \cdot \nabla r \\ &= \Phi'(r) \cdot \frac{x - \bar{x}}{\|x - \bar{x}\|} \\ &= \Phi'(r) \cdot \frac{x - \bar{x}}{r}.\end{aligned}$$

We define the radial function:

$$f(r) := \frac{1}{r}\frac{d\Phi(r)}{dr},$$

so that the gradient becomes:

$$\nabla F(x) = f(r)(x - \bar{x}).$$

### Step-by-step breakdown

Write the norm as a square root of an inner product:

$$r(x) = \|x - \bar{x}\| = \sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Differentiate using the chain rule:

**Note:** Since $r(x)$ is a scalar-valued function of a single vector variable $x$, computing the gradient of $r$ is equivalent to differentiating with respect to $x$.

$$\frac{d}{dx}\sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Let's set $u(x) = (x - \bar{x})^\top (x - \bar{x})$. Then $r(x) = \sqrt{u(x)}$, and we use the chain rule:

$$\frac{d}{dx} r(x) = \frac{1}{2\sqrt{u(x)}} \cdot \frac{d}{dx} u(x)$$

Differentiate the inner function:

$$\frac{d}{dx}\left((x - \bar{x})^\top (x - \bar{x})\right) = 2(x - \bar{x})$$

$$\frac{d}{dx}\|x - \bar{x}\| = \frac{1}{2\|x - \bar{x}\|} \cdot 2(x - \bar{x}) = \frac{x - \bar{x}}{\|x - \bar{x}\|}$$

### Clarifying the Derivative Step

We are differentiating:

$$r(x) = \|x - \bar{x}\| = \sqrt{(x - \bar{x})^\top (x - \bar{x})}$$

Let's denote:

$$u(x) = (x - \bar{x})^\top (x - \bar{x}) \quad \text{so} \quad r(x) = \sqrt{u(x)}$$

$$\frac{d}{dx}\sqrt{u(x)} = \frac{1}{2\sqrt{u(x)}} \cdot \frac{du(x)}{dx}$$

```python
def _radial_f_prime(self, r, alpha):
    """
    The derivative of the radial localization function, f'(r).
    Handles the r=0 case, where the derivative is 0.

    Args:
        r (np.ndarray): Array of radial distances ||x - x0||.
        alpha (float): The length-scale of the map.

    Returns:
        np.ndarray: The value of the derivative.
    """
    f_prime_vals = np.zeros_like(r)
    nonzero_r = r != 0

    r_nz = r[nonzero_r]
    r_scaled = r_nz / alpha

    term1 = (2.0 / (alpha * np.sqrt(np.pi))) * np.exp(-r_scaled**2)
    term2 = erf(r_scaled) / r_nz
    f_prime_vals[nonzero_r] = (term1 - term2) / r_nz

    return f_prime_vals
```

# Mathematical Derivation of _radial_f_prime

We start with the *radial localization function*

$$f(r) = \frac{\operatorname{erf}\left(\frac{r}{\alpha}\right)}{r}, \quad r > 0,$$

where:

- $r = \|x - x_0\|$ is the Euclidean distance from $x$ to the center $x_0$,
- $\alpha > 0$ is the length–scale of the map,
- erf is the Gauss error function:

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt.$$

## Differentiate $f(r)$ for $r > 0$

Let

$$g(r) = \operatorname{erf}\left(\frac{r}{\alpha}\right),$$

so that

$$g'(r) = \frac{2}{\alpha\sqrt{\pi}} e^{-\left(\frac{r}{\alpha}\right)^2}.$$

Using the quotient rule:

$$f'(r) = \frac{g'(r) \cdot r - g(r)}{r^2}.$$

## Substitute $g$ and $g'$

We obtain:

$$f'(r) = \frac{\frac{2}{\alpha\sqrt{\pi}} e^{-\left(\frac{r}{\alpha}\right)^2} - \frac{\operatorname{erf}\left(\frac{r}{\alpha}\right)}{r}}{r}.$$

## Handle $r = 0$.

From the Taylor expansion of $\operatorname{erf}(x)$ around $x = 0$, we have

$$f(r) \approx \frac{2}{\alpha\sqrt{\pi}} - \frac{2}{3\alpha^3\sqrt{\pi}} r^2 + O(r^4),$$

so

$$f'(0) = 0.$$

The code explicitly sets $f'(0) = 0$ to avoid division by zero.

## Final formula implemented in code

For $r > 0$:

$$f'(r) = \frac{\frac{2}{\alpha\sqrt{\pi}} e^{-\left(\frac{r}{\alpha}\right)^2} - \frac{\operatorname{erf}\left(\frac{r}{\alpha}\right)}{r}}{r}.$$

For $r = 0$:

$$f'(0) = 0.$$

```
def fit(self, x, log_rho_exact=None, n_steps=1000):
    """
    Fits the density model to the data by building a sequence of maps.
    """
    m, n = x.shape
    tol = 1e-9 # Tolerance for Hessian

    # Starting map
    mean = np.mean(x, axis=0)
    z = x - mean
    std = np.sqrt(np.mean(np.sum(z**2, axis=1)) / n)
    z /= std

    self.preconditioning = {'mean': mean, 'std': std}
    self.maps = []

    # Iterative map building
    for i in range(n_steps):
        # 1. Select a center x0
        if np.random.rand() > 0.5:
            # Pick from the actual observations at their current normalized state
            idx = np.random.randint(m)
            x0 = z[idx]
        else:
            # Sample the target normal distribution to keep x0 from becoming too large
            x0 = np.random.randn(n)

        # 2. Calculate length-scale alpha
        alpha = self._calculate_alpha(x0, n, m)

        # 3. Calculate Gradient (G) and Hessian (H) of log-likelihood
        r = np.linalg.norm(z - x0, axis=1)
        f = self._radial_f(r, alpha)
        f_prime = self._radial_f_prime(r, alpha)

        term_G = n + np.dot(z, x0) - np.sum(z**2, axis=1)
        G = np.sum(term_G * f + r * f_prime)

        term_H1 = (n + r**2) * f**2
        term_H2 = 2 * r * f * f_prime
        term_H3 = (r * f_prime)**2
        H = -np.sum(term_H1 + term_H2 + term_H3)

        if np.abs(H) < tol:
            continue

        # 4. Calculate optimal step beta and constrain it
        beta = -G / H
        beta = np.clip(beta, -self.epsilon, self.epsilon)

        # 5. Apply the map to transform the data
        z = z + beta * f[:, np.newaxis] * (z - x0)

        # 6. Store the map's parameters
        self.maps.append({'x0': x0, 'alpha': alpha, 'beta': beta})

        if log_rho_exact is not None:
            # Estimate the KL divergence if the exact log density is provided
```

```
            kl = estimate_forward_KL(x, self, log_rho_exact)
            self.kl_history.append(kl)

        if (i+1) % (n_steps//10) == 0:
            if log_rho_exact is None:
                print(f"Step {i+1}/{n_steps} completed")
            else:
                kl = estimate_forward_KL(x, self, log_rho_exact)
                print(f"Step {i+1}/{n_steps} completed, KL estimate: {kl:.4f}")
```

# Mathematical Explanation of `fit`

## Setup (preconditioning)

Given samples $x_1, \dots, x_m \in \mathbb{R}^n$ from $\rho$, define

$$\bar{x} = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad z_i^{(0)} = \frac{x_i - \bar{x}}{\text{std}}, \quad \text{std} = \sqrt{\frac{1}{mn}\sum_{i=1}^{m} \|x_i - \bar{x}\|^2}.$$

We now work with $z \equiv z^{(0)}$ (mean 0, isotropic scale). mn gives the average variance per coordinate.

## One Elementary Map

Pick a center $x_0 \in \mathbb{R}^n$ (either a data point $z_j$ or a standard normal draw), choose a length scale $\alpha > 0$ (by the small-ball rule), and define for a scalar parameter $\beta \in \mathbb{R}$ the *radial* update

$$T_\beta(z) = z + \beta\, f(r)\,(z - x_0), \qquad r = \|z - x_0\|,$$

with

$$f(r) = \frac{\operatorname{erf}(r/\alpha)}{r}, \qquad f'(r) = \frac{\frac{2}{\alpha\sqrt{\pi}} e^{-(r/\alpha)^2} - \frac{\operatorname{erf}(r/\alpha)}{r}}{r}.$$

## Log–likelihood for a Standard Gaussian Target

For the standard normal base $\mu(y) = (2\pi)^{-n/2}\exp(-\|y\|^2/2)$, the log–likelihood of the transformed cloud $\{y_i = T_\beta(z_i)\}$ is

$$\mathcal{L}(\beta) = \sum_{i=1}^{m} \Big( \log \mu(T_\beta(z_i)) + \log |\det \nabla T_\beta(z_i)| \Big).$$

## Jacobian Eigenvalues of a Radial Map

This equation can be found in the Tabak–Turner paper (page 11):

$$\log |\det \nabla T_\beta| = (n-1)\log\big(1 + \beta f\big) + \log\big(1 + \beta(f + r f')\big).$$

## Quadratic Expansion in $\beta$

Expand $\mathcal{L}(\beta)$ at $\beta = 0$ to second order:

$$\mathcal{L}(\beta) \approx \mathcal{L}(0) + G\,\beta - \tfrac{1}{2} H\,\beta^2,$$

where $G$ and $H$ are given in the Tabak–Turner paper, page 12:

$$G = \sum_{i=1}^{m} \Big[ (n + z_i \cdot x_0 - \|z_i\|^2) f(r_i) \; + \; r_i f'(r_i) \Big],$$

$$H = \sum_{i=1}^{m} \Big[ (n + r_i^2) f(r_i)^2 \; + \; 2 r_i f(r_i) f'(r_i) \; + \; (r_i f'(r_i))^2 \Big].$$

### Newton Step for the Optimal Strength

The quadratic approximation

$$\mathcal{L}(\beta) \approx \mathcal{L}(0) + G\,\beta - \tfrac{1}{2} H\,\beta^2$$

is a concave parabola in $\beta$ (when $H > 0$). Maximizing this requires setting its derivative with respect to $\beta$ to zero:

$$\frac{d}{d\beta}\mathcal{L}(\beta) \approx G - H\,\beta = 0.$$

Solving for $\beta$ gives

$$\beta^\star \approx \frac{G}{H}.$$

This is exactly the Newton–Raphson step for maximizing $\mathcal{L}$ using the first and second derivatives evaluated at $\beta = 0$. For stability, the resulting step is *clipped* to remain within the bounds $\beta^\star \in [-\varepsilon, \varepsilon]$, meaning

$$\beta^\star \;\leftarrow\; \begin{cases} \varepsilon, & \text{if } \beta^\star > \varepsilon, \\ -\varepsilon, & \text{if } \beta^\star < -\varepsilon, \\ \beta^\star, & \text{otherwise.} \end{cases}$$

This prevents excessively large updates, which could violate the smoothness assumptions of the quadratic approximation and cause instability in the flow.

### Apply and Compose

Update all particles

$$z_i \leftarrow z_i + \beta^\star f(r_i)\,(z_i - x_0),$$

store the triplet $(x_0, \alpha, \beta^\star)$, and repeat the procedure for $k = 1, \ldots, n_{\text{steps}}$ to build a composition of elementary maps. (Optionally, estimate the forward KL to a known $\log \rho$ for monitoring.)

**Do Only Nearby Points Get Pushed by the Gradient Update?**

- The gradient $\nabla F_\ell(z)$ is nonzero everywhere, but its strength depends on the distance from the center $\tilde{z}_\ell$.
- It grows rapidly near $\tilde{z}_\ell$, then flattens as $\|z - \tilde{z}_\ell\| \gg \alpha$.
- Faraway points receive nearly constant, directionless pushes — effectively contributing nothing to transport.
- Thus, although the field has global support, it acts locally: only points within distance $\sim \alpha$ are meaningfully updated.

## Intuition Behind the Objective

The goal is to choose the flow parameters $\beta$ so that the transformed points look as if they were drawn from a standard Gaussian distribution. This is achieved by **maximizing the log-likelihood** of the transformed points under the Gaussian model, which is equivalent to **minimizing the Kullback–Leibler divergence**.

## Likelihood as "How Likely Are My Points Under This Model?"

Suppose we observe points $z_1, \ldots, z_m$ and assume they come from some model density $p_\beta(z)$, where $\beta$ controls the shape of the model (in our case, the parameters of the flow).

The *likelihood* is:

$$L(\beta) = \prod_{i=1}^{m} p_\beta(z_i),$$

and the *log-likelihood* is:

$$\mathcal{L}(\beta) = \sum_{i=1}^{m} \log p_\beta(z_i).$$

Here:

- A large $\mathcal{L}(\beta)$ means the model assigns high probability to the observed data $\Rightarrow$ good fit.
- A small $\mathcal{L}(\beta)$ means the model assigns low probability $\Rightarrow$ bad fit.

## Connection to the Flow Setting

For our flow, the change-of-variables formula gives:

$$p_\beta(z) = \mu\big(T_\beta(z)\big)\ \left|\det \nabla T_\beta(z)\right|,$$

where $\mu$ is the standard Gaussian density. Thus:

$$\log p_\beta(z) = \log \mu\big(T_\beta(z)\big) + \log \left|\det \nabla T_\beta(z)\right|.$$

Maximizing $\mathcal{L}(\beta)$ therefore means choosing $\beta$ so that the transformed points $T_\beta(z_i)$ look Gaussian *and* the transformation has the correct volume change (Jacobian determinant).

## Why Flowing Target $\to$ Gaussian Doesn't Blow Up KL

The forward KL can be written as:

$$\mathrm{KL}(\rho_{\text{exact}} \,\|\, \rho_{\text{flow}}) = \mathbb{E}_{x\sim\rho_{\text{exact}}} \left[\log \rho_{\text{exact}}(x) - \log \rho_{\text{flow}}(x)\right].$$

The first term $\log \rho_{\text{exact}}(x)$ is constant for our dataset. Thus, minimizing KL is equivalent to maximizing:

$$\mathbb{E}_{x\sim\rho_{\text{exact}}} \left[\log \rho_{\text{flow}}(x)\right],$$

which, using the change-of-variables expression, becomes:

$$\mathbb{E}_{x\sim\rho_{\text{exact}}} \left[\log \mu(T(x)) + \log \left|\det \nabla T(x)\right|\right].$$

If $T$ is a good map, $T(x) \sim \mathcal{N}(0, I)$ and $\log \mu(T(x))$ is maximized, making the KL small. If $T$ is a bad map, the transformed points do not look Gaussian, $\log \mu(T(x))$ is small, and KL is large.

```python
def _transform(self, x_new):
    """Applies the full sequence of learned maps to new data."""
    if x_new.ndim == 1:
        x_new = x_new.reshape(1, -1)
    m, n = x_new.shape

    # Apply preconditioning
    z = (x_new - self.preconditioning['mean']) / self.preconditioning['std']

    # Calculate initial log Jacobian from preconditioning
```

```
    log_J = np.full(m, -n * np.log(self.preconditioning['std']))

    # Apply sequence of maps
    for p in self.maps:
        x0, alpha, beta = p['x0'], p['alpha'], p['beta']

        r = np.linalg.norm(z - x0, axis=1)
        f = self._radial_f(r, alpha)
        f_prime = self._radial_f_prime(r, alpha)

        # Update log Jacobian determinant
        term1 = 1 + beta * f
        term2 = 1 + beta * (f + r * f_prime)

        valid_map = (term1 > 0) & (term2 > 0)

        log_J_update = np.zeros(m)
        if np.any(valid_map):
            log_J_update[valid_map] = (n - 1) * np.log(term1[valid_map]) + np.log(term2[valid_map])
        log_J += log_J_update

        # Apply the map
        z = z + beta * f[:, np.newaxis] * (z - x0)
```

# Mathematical explanation of `_transform`

## Goal

Given *new* points $x^{\text{new}} \in \mathbb{R}^n$ (not the training data $x$ used in `fit`), the function applies the *full composition* of the learned radial maps from training, while also keeping track of the log–determinant of the Jacobian of the transformation. The parameters $(x_0, \alpha, \beta)$ for each map are *exactly those learned during training* and stored in `self.maps`; no new $x_0$ is sampled at this stage.

## Why Preconditioning Happens Again

During training (`fit`), the dataset $x$ was preconditioned (mean-centered and rescaled) before fitting the maps. The resulting mean $\mu_{\text{train}}$ and scale $\sigma_{\text{train}}$ were stored. When transforming *new* data $x^{\text{new}}$, we must bring it into the same normalized coordinate system as the training data before applying the learned maps. Thus the preconditioning step here:

$$z^{(0)} = \frac{x^{\text{new}} - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

is applied only once to the new points — it does *not* "double precondition" the training data.

## Initial Log–Jacobian from Preconditioning

Since preconditioning is an affine rescaling, the initial log–determinant is

$$\log J^{(0)} = -n \log \sigma_{\text{train}}.$$

## Applying Each Learned Map

Each stored map is parameterized by $(x_0, \alpha, \beta)$, where $x_0$ is the fixed center selected during training for that map. For the current $z \in \mathbb{R}^n$:

**Compute radial distance:**

$$r = \|z - x_0\|.$$

**Evaluate radial profile and derivative:**

$$f(r) = \frac{\operatorname{erf}(r/\alpha)}{r}, \quad f'(r) = \frac{\frac{2}{\alpha\sqrt{\pi}} e^{-(r/\alpha)^2} - \frac{\operatorname{erf}(r/\alpha)}{r}}{r}.$$

**Jacobian eigenvalues of the radial map:** From the Tabak–Turner formula (p. 11),

$$\lambda_{\tan}(r) = 1 + \beta f(r) \quad \text{(multiplicity } n-1\text{)},$$

$$\lambda_{\mathrm{rad}}(r) = 1 + \beta\big(f(r) + r f'(r)\big).$$

**Update log–Jacobian:** If both $\lambda_{\tan}$ and $\lambda_{\mathrm{rad}}$ are positive (ensuring local invertibility), the log–determinant increment is:

$$\Delta \log J = (n-1)\log\big(1 + \beta f(r)\big) + \log\big(1 + \beta\big(f(r) + r f'(r)\big)\big),$$

and we update:

$$\log J \leftarrow \log J + \Delta \log J.$$

**Update $z$:** Apply the radial map:

$$z \leftarrow z + \beta f(r)\,(z - x_0).$$

**Result:** After all maps are applied, $z$ contains the transformed samples in the target space, and $\log J$ contains the accumulated log–determinant of the Jacobian for the overall transformation.

```python
def log_prob(self, x_new):
    """
    Calculates the log probability density log(rho(x)) for new data points.

    Args:
        x_new (np.ndarray): New data points, shape (m, n).

    Returns:
        np.ndarray: The log probability for each point.
    """
    m, n = x_new.shape

    # Transform data to the target distribution space (y) and get log Jacobian
    y, log_J = self._transform(x_new)

    # Calculate log probability in the target Gaussian space
    log_prob_gaussian = -0.5 * np.sum(y**2, axis=1) - 0.5 * n * np.log(2 * np.pi)

    # Final log probability is log(rho(x)) = log(mu(y(x))) + log(J(x))
    return log_prob_gaussian + log_J
```

# Mathematical explanation of `log_prob`

## Change of Variables

The model density is defined by

$$\rho_{\mathrm{flow}}(x) \;=\; \mu\big(T(x)\big)\;\big|\det \nabla T(x)\big|\,.$$

Taking logs,

$$\log \rho_{\mathrm{flow}}(x) \;=\; \log \mu\big(T(x)\big) \;+\; \log\big|\det \nabla T(x)\big|\,.$$

### Quantities Computed by `_transform`

Calling `_transform`$(x)$ produces

$$(y,\ \log J)\ =\ \texttt{\_transform}(x), \qquad y\ =\ T(x), \quad \log J\ =\ \log|\det \nabla T(x)|\,.$$

The term $\log J$ is accumulated in `_transform` and passed here unchanged.

### Gaussian Base

For the standard Gaussian base

$$\mu(y) = (2\pi)^{-n/2} \exp\!\left(-\tfrac{1}{2}\|y\|^2\right),$$

we have

$$\log \mu(y)\ =\ -\tfrac{1}{2}\|y\|^2\ -\ \tfrac{n}{2}\log(2\pi).$$

### Final Expression Returned by `log_prob`

For each sample $x_i$ with $y_i = T(x_i)$ and $\log J_i$ from `_transform`,

$$\log \rho_{\text{flow}}(x_i)\ =\ -\tfrac{1}{2}\|y_i\|^2\ -\ \tfrac{n}{2}\log(2\pi)\ +\ \log J_i$$

and the function returns the vector

$$\big(\log \rho_{\text{flow}}(x_1), \ldots, \log \rho_{\text{flow}}(x_m)\big).$$

**Note:** This function is called inside `estimate_forward_KL` to provide the per-sample $\log \rho_{\text{flow}}(x)$ values used in the KL divergence calculation.

```python
def sample(self, n_samples):
    """
    Generates samples from the learned density model.

    Args:
        n_samples (int): Number of samples to generate.

    Returns:
        np.ndarray: Generated samples, shape (n_samples, n).
    """
    # Sample from the standard normal distribution
    y_samples = np.random.randn(n_samples, len(self.maps[0]['x0']))

    # Apply the inverse of the learned maps to get samples in the original space
    z_samples = y_samples.copy()

    for p in reversed(self.maps):
        x0, alpha, beta = p['x0'], p['alpha'], p['beta']
        r = np.linalg.norm(z_samples - x0, axis=1)
        f = self._radial_f(r, alpha)

        # Apply the inverse map
        z_samples = z_samples - beta * f[:, np.newaxis] * (z_samples - x0)

    # Apply preconditioning to get back to original space
    return z_samples * self.preconditioning['std'] + self.preconditioning['mean']
```

# Mathematical Explanation of `sample`

## Draw from the *target* $\mu$

Sample i.i.d.

$$y^{(i)} \sim \mu = \mathcal{N}(0, I_n), \qquad i = 1, \ldots, N,$$

forming $Y_{\text{samples}} \in \mathbb{R}^{N \times n}$.

## Apply the inverse learned map in the normalized (preconditioned) space

During training, data were preconditioned to

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

and then mapped to the *target* via the composition

$$T \;=\; T_L \circ \cdots \circ T_1, \qquad T_\ell(u) = u + \beta_\ell \, f_\ell(r) \, (u - x_{0,\ell}), \quad r = \|u - x_{0,\ell}\|.$$

To synthesize $z$ from $y$, apply inverses in reverse order:

$$z \;=\; T^{-1}(y) \;=\; T_1^{-1} \circ \cdots \circ T_L^{-1}(y),$$

with each inverse used in the code as

$$T_\ell^{-1}(u) = u - \beta_\ell \, f_\ell(r) \, (u - x_{0,\ell}), \qquad r = \|u - x_{0,\ell}\|.$$

## Undo Preconditioning to Return to $x$-space

$$x_{\text{gen}} \;=\; z \, \sigma_{\text{train}} \;+\; \mu_{\text{train}}.$$

Now, the samples $x_{\text{gen}}$ lie in the *source* space and are distributed according to the model's approximation $\rho_{\text{flow}}$ to $\rho$. Here the *target* $\mu$ supplies randomness; $T^{-1}$ transports it back to the source coordinates.

```python
import numpy as np

def sample_exact(n_samples):
    theta = np.random.randn(n_samples)
    r     = 1.0 + 0.1 * np.random.randn(n_samples)
    x = r * np.cos(theta)
    y = r * np.sin(theta)
    return np.stack([x, y], axis=1)

def log_rho_exact(xy):
    x = xy[:,0]
    y = xy[:,1]
    r     = np.sqrt(x**2 + y**2)
    theta = np.arctan2(y, x)

    log_p_theta = -0.5*theta**2 - 0.5*np.log(2*np.pi)

    log_p_r = (
        -0.5*((r - 1.0)/0.1)**2
        - 0.5*np.log(2*np.pi*0.1**2)
    )

    # log Jacobian term
    log_jac = -np.log(r)

    return log_p_theta + log_p_r + log_jac
```

# Mathematical Explanation of `sample_exact` and `log_rho_exact`

The function `sample_exact` generates $n_{\text{samples}}$ i.i.d. points $(x, y) \in \mathbb{R}^2$ from a distribution defined in *polar coordinates* $(r, \theta)$.

## Distribution in Polar Coordinates

We define the radius $r$ and angle $\theta$ as

$$\theta \sim \mathcal{N}(0, 1), \quad r \sim \mathcal{N}(\mu_r, \sigma_r^2), \quad \mu_r = 1.0, \quad \sigma_r = 0.1.$$

The mean radius $\mu_r = 1.0$ ensures that, on average, samples lie one unit away from the origin. In Cartesian coordinates, this corresponds to points clustered around the circumference of a circle of radius 1. The small standard deviation $\sigma_r = 0.1$ means that roughly 68% of the points will fall within the radial band $[0.9, 1.1]$, producing a thin annulus rather than a thick disk. Geometrically, this creates a ring-shaped cloud with a controlled radial "thickness" determined by $\sigma_r$.

For the angular component, $\theta$ is drawn from a standard normal distribution $\mathcal{N}(0, 1)$ rather than from a uniform distribution on $[0, 2\pi)$. This choice means that the majority of points have angles close to $\theta = 0$, with the density decreasing as $|\theta|$ increases. In Cartesian space, this produces a ring in which points are most densely clustered along the positive $x$-axis direction (corresponding to $\theta \approx 0$) and become sparser as we move around the circle. The spread in $\theta$ controls how far this high-density region extends, effectively determining how much of the ring is populated. By introducing this directional bias, we can test whether the density estimation method is capable of recovering both the radial structure (the annulus) and the non-uniform angular distribution.

## Sampling Procedure

Given $n_{\text{samples}}$, the algorithm:

- Draws $\theta_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$ and $r_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(1.0, 0.1^2)$ for $i = 1, \ldots, n_{\text{samples}}$.
- Converts $(r_i, \theta_i)$ to Cartesian coordinates via

$$x_i = r_i \cos\theta_i, \quad y_i = r_i \sin\theta_i.$$

- Returns the matrix

$$\mathbf{X} = \begin{bmatrix} x_1 & y_1 \\ \vdots & \vdots \\ x_{n_{\text{samples}}} & y_{n_{\text{samples}}} \end{bmatrix} \in \mathbb{R}^{n_{\text{samples}} \times 2}.$$

## Exact Log-Density

The function `log_rho_exact` computes the exact log-probability density $\log \rho(x, y)$ for points drawn from this distribution.

First, given a point $(x, y)$, we recover polar coordinates:

$$r = \sqrt{x^2 + y^2}, \quad \theta = \text{atan2}(y, x).$$

### `atan2(y, x)`

The function $\text{atan2}(y, x)$ returns the polar angle $\theta$ of the point $(x, y)$ in the plane. Unlike the standard arctangent $\arctan(y/x)$, which is restricted to $(-\pi/2, \pi/2)$ and cannot distinguish between points in different quadrants, atan2 uses the signs of both $x$ and $y$ to determine the correct quadrant. It returns values in

$(-\pi, \pi]$, covering the entire circle without discontinuities at the $\pm\pi$ boundary.

For example, the points $(1, 1)$ and $(-1, -1)$ both satisfy $y/x = 1$, so $\arctan(y/x) = \pi/4$ in both cases. However, $\text{atan2}(1, 1) = \pi/4$ (first quadrant) while $\text{atan2}(-1, -1) = -3\pi/4$ (third quadrant), correctly distinguishing their angular positions. This makes atan2 especially useful for converting Cartesian coordinates $(x, y)$ to polar coordinates $(r, \theta)$ in a continuous and unambiguous way.

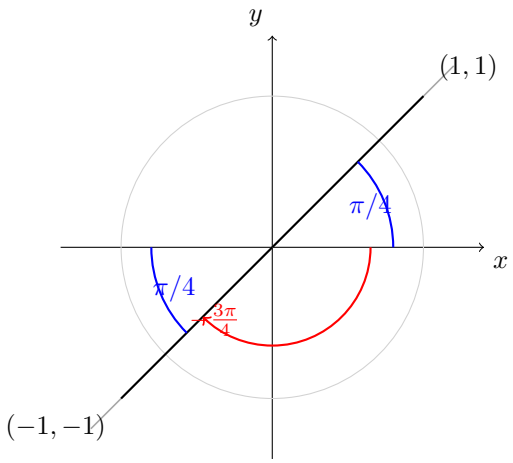


Figure 1: Arctan ambiguity: $\arctan(y/x) = \pi/4$ for both $(1, 1)$ and $(-1, -1)$ because they share slope 1. Quadrant-aware atan2 gives $\text{atan2}(1, 1) = \pi/4$ and $\text{atan2}(-1, -1) = -3\pi/4$, where $-3\pi/4 = 225°$ is the same direction in a different convention.

The joint density in polar coordinates factorizes as

$$p(r, \theta) = p_r(r)\, p_\theta(\theta),$$

where

$$p_\theta(\theta) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{\theta^2}{2}\right), \quad p_r(r) = \frac{1}{\sqrt{2\pi\sigma_r^2}} \exp\left(-\frac{(r-\mu_r)^2}{2\sigma_r^2}\right).$$

## Jacobian Correction

Transforming from polar to Cartesian coordinates requires dividing by the Jacobian determinant

$$\left|\frac{\partial(r, \theta)}{\partial(x, y)}\right| = \frac{1}{r}.$$

Thus, the Cartesian density is

$$\rho(x, y) = \frac{p_r(r)\, p_\theta(\theta)}{r}.$$

Taking logs yields

$$\log \rho(x, y) = \log p_\theta(\theta) + \log p_r(r) - \log r.$$

This is exactly the expression computed in `log_rho_exact`.

```
x = sample_exact(1000)  # Sample from the true target density

flow = NormalizingFlow(n_p=500, epsilon=0.1)

n_steps = 600  # Number of steps to fit the model
flow.fit(x=x, log_rho_exact=log_rho_exact, n_steps=n_steps)
```

`sample_exact(1000)` calls the `sample_exact` function defined earlier, producing 1000 samples from the true target density. These samples are passed as input data to the training routine. `NormalizingFlow(n_p=500, epsilon=0.1)` creates a new instance of the `NormalizingFlow` class defined earlier in the code, with `n_p=500` setting the number of points each local radial map is designed to influence and `epsilon=0.1` setting the maximum allowed absolute step size for the parameter $\beta$ in each map update. `n_steps = 600` specifies the number of gradient update iterations to run in the `fit` method. Finally, `flow.fit(...)` calls the `fit` method of the `NormalizingFlow` instance, where `x` is the array returned by `sample_exact`, `log_rho_exact` is the exact log-density function defined earlier in the program, and `n_steps` controls the number of update iterations. Inside `fit`, the method may call `estimate_forward_KL` to monitor the KL divergence between the learned flow density and the exact target density.

```
samples = flow.sample(1000)  # Generate samples from the learned density model

plt.scatter(samples[:, 0], samples[:, 1], s=1, alpha=0.5, label='Sampled Points')
plt.scatter(x[:, 0], x[:, 1], s=1, alpha=0.5, color='red', label='Original Points')
plt.title('Samples from the Learned Density Model')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
```

## Sampling and Visualizing the Learned Distribution

This section of the code generates and visualizes new samples from the learned normalizing flow. First, `samples = flow.sample(1000)` draws 1000 points by sampling from the standard normal base distribution in the flow's latent space and applying the inverse of the learned transformation maps to return to the original data space. The `plt.scatter` command then plots these generated samples in blue, while the second `plt.scatter` plots the original target distribution points in red. By visually comparing the two sets of points, we can assess how effectively the learned flow transforms a standard normal distribution into the target distribution.

```
plt.plot(flow.kl_history)
plt.title('KL Divergence History')
plt.xlabel('Step')
plt.ylabel('KL')
plt.yticks([0, 0.5, 1.0, 1.5])
```

### Plotting KL Divergence History

This code visualizes the evolution of the Kullback–Leibler (KL) divergence during training. It plots the stored history of KL values (`flow.kl_history`) over the optimization steps, providing a direct way to assess how effectively the flow reduces the divergence between the learned distribution and the target distribution. A decreasing curve indicates successful training progress.

```
# Test Case 1
# This creates a crescent-shaped distribution, similar to the paper
m = 1000
angle = np.random.uniform(-np.pi/2, np.pi/2, m)
radius = 1.0 + 0.1 * np.random.randn(m)
x = np.zeros((m, 2))
x[:, 0] = radius * np.cos(angle)
x[:, 1] = radius * np.sin(angle)
x += 0.02 * np.random.randn(m, 2)
```

```
# Train the model
# Using parameters similar to the paper.
# n_p = 50
# n_steps = 500.
model = NormalizingFlow(n_p=50, epsilon=0.5)
model.fit(x, n_steps=600)
```

# Mathematical Explanation of the Crescent Distribution Generation

This test case creates $m = 1000$ two-dimensional points $(x, y) \in \mathbb{R}^2$ forming a crescent-shaped distribution. The shape is generated in *polar coordinates* $(r, \theta)$, then converted to Cartesian coordinates and perturbed with Gaussian noise.

## Distribution in Polar Coordinates

The angle $\theta$ is sampled uniformly from a half-circle:

$$\theta \overset{\text{i.i.d.}}{\sim} \mathcal{U}\left(-\frac{\pi}{2}, \frac{\pi}{2}\right).$$

This restriction ensures that all points lie in the right-hand side of the plane, covering only 180° rather than the full circle. It is the key factor that creates the open, crescent-like geometry. The radius $r$ is sampled from a normal distribution centered at 1.0 with small variance:

$$r \overset{\text{i.i.d.}}{\sim} \mathcal{N}(1.0, 0.1^2).$$

This means that most points will lie close to the unit circle, with a radial "thickness" controlled by $\sigma_r = 0.1$.

## Conversion to Cartesian Coordinates

Given $(r_i, \theta_i)$, each point is mapped to Cartesian coordinates via

$$x_i = r_i \cos\theta_i, \quad y_i = r_i \sin\theta_i.$$

The result is a semi-circular arc of points centered around $(0, 0)$ with radius $\approx 1.0$.

## Adding Local Noise

To avoid a perfectly smooth curve and to better approximate real-world noisy data, we add independent Gaussian perturbations to both coordinates:

$$x_i \leftarrow x_i + \epsilon_{x,i}, \quad y_i \leftarrow y_i + \epsilon_{y,i},$$

where

$$\epsilon_{x,i}, \epsilon_{y,i} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 0.02^2).$$

This step introduces small variations in both radial and tangential directions, giving the crescent a more scattered appearance.

```
# Results
# Create a grid of points to evaluate the learned density
grid_res = 1000
x_range = np.linspace(-1.5, 1.5, grid_res)
y_range = np.linspace(-1.5, 1.5, grid_res)
xx, yy = np.meshgrid(x_range, y_range)
```

```
grid_points = np.c_[xx.ravel(), yy.ravel()]

# Calculate the probability on the grid
log_p = flow.log_prob(grid_points)
p = np.exp(log_p).reshape(grid_res, grid_res)

# Plotting
plt.style.use('seaborn-v0_8-whitegrid')
fig = plt.figure(figsize=(12, 5))

# Plot 1: Original Data
ax1 = fig.add_subplot(1, 2, 1)
ax1.scatter(x[:, 0], x[:, 1], s=10, alpha=0.6, c='blue')
ax1.set_title("Original Data Sample (m=1000)")
ax1.set_xlabel("x1")
ax1.set_ylabel("x2")
ax1.set_aspect('equal', adjustable='box')
ax1.set_xlim(-1.5, 1.5)
ax1.set_ylim(-1.5, 1.5)

# Plot 2: Learned Density
ax2 = fig.add_subplot(1, 2, 2)
# Use LogNorm for better visualization of low-density areas
contour = ax2.contourf(xx, yy, p, levels=15, cmap='viridis', norm=LogNorm())
ax2.contour(xx, yy, p, levels=15, colors='white', linewidths=0.5, alpha=0.7)
fig.colorbar(contour, ax=ax2, label="Probability Density")
ax2.set_title("Learned Density with Normalizing Flow")
ax2.set_xlabel("x1")
ax2.set_ylabel("x2")
ax2.set_aspect('equal', adjustable='box')
ax2.set_xlim(-1.5, 1.5)
ax2.set_ylim(-1.5, 1.5)

plt.tight_layout()
plt.show()
```

## Evaluating and Visualizing the Learned Density

This section evaluates the Normalizing Flow by testing it on a set of evenly distributed points across the domain and comparing the resulting learned density to the original data.

- An evenly spaced grid of points is created across the region $[-1.5, 1.5] \times [-1.5, 1.5]$.
- The trained model is applied to each grid point to estimate the probability density at that location.
- Those likelihood values are then arranged back into the grid's shape so that they can be plotted as a smooth heatmap of the learned density.
- Two plots are generated: one showing the original data samples and another showing the learned density field.
- Using an evenly spaced grid ensures that the learned distribution is evaluated uniformly across the entire domain, allowing for a fair visual comparison to the true data distribution.